

# SAS 巨集

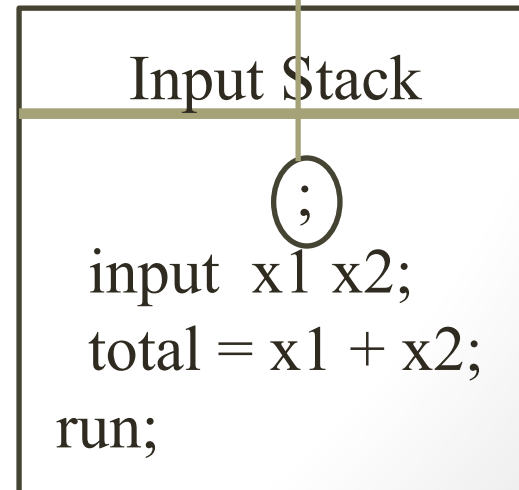
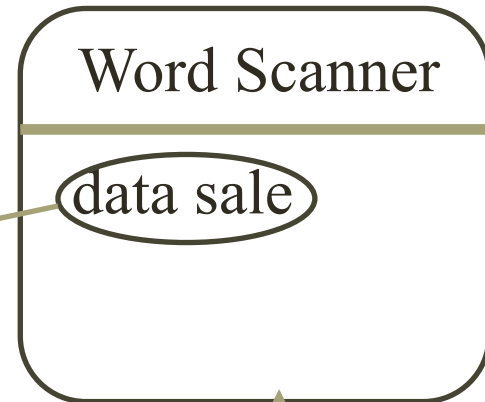
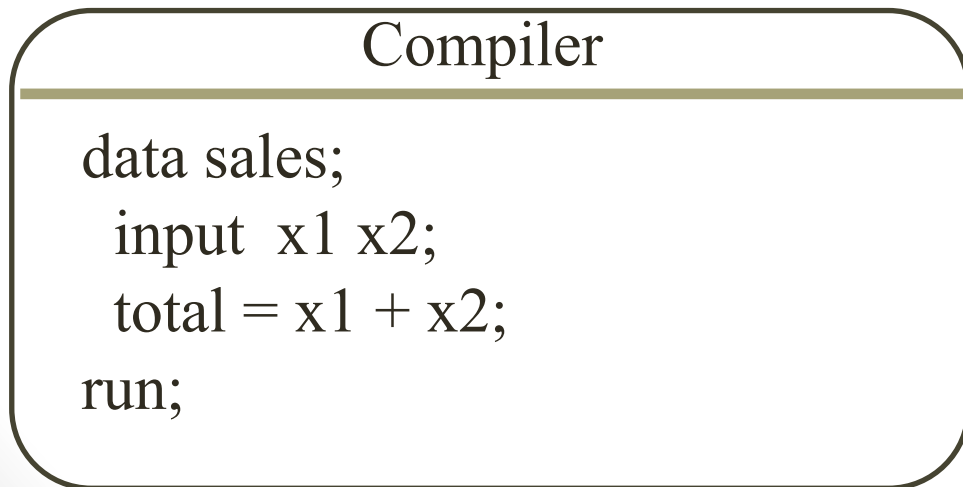
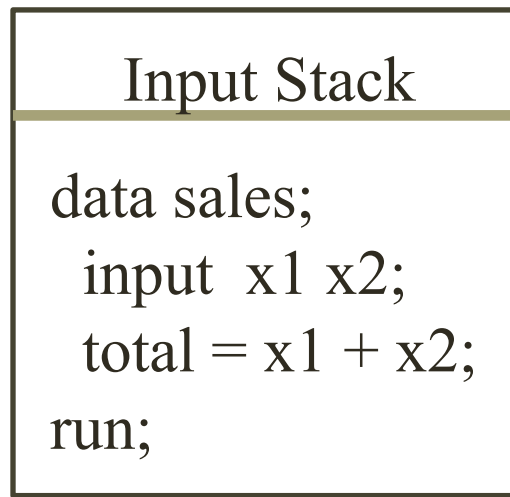
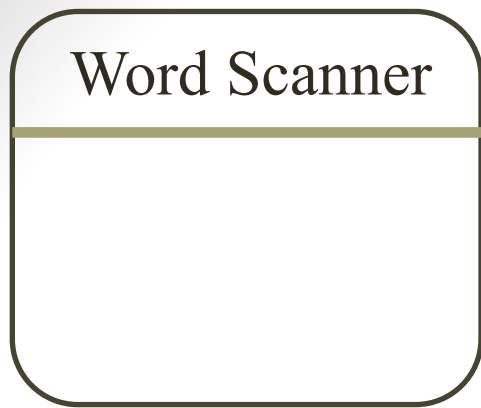
YI-TING HWANG  
DEPARTMENT OF STATISTICS  
NATIONAL TAIPEI UNIVERSITY

# 目的

- 延伸並個人化 SAS 系統
- 重複產出所需執行的工作

# SAS 程式執行流程

- 標記 (Tokenization )
  - SAS 從 input stack 擷取 word 或 symbol 的過程
- 文字辨識 (Word scanner)
  - 辨識標記的方式
  - 從 input stack 的第一個字逐一辨識
  - 在轉換成 token (詞元)
- Token 記錄的數據型態
  - Literal – 包含引號的字串
  - Number – 數值
  - Name – 變數名稱，由底線或是字母開始的一個序列
  - Special – 包含特殊符號



# SAS 執行巨集方式

Compiler

Word Scanner

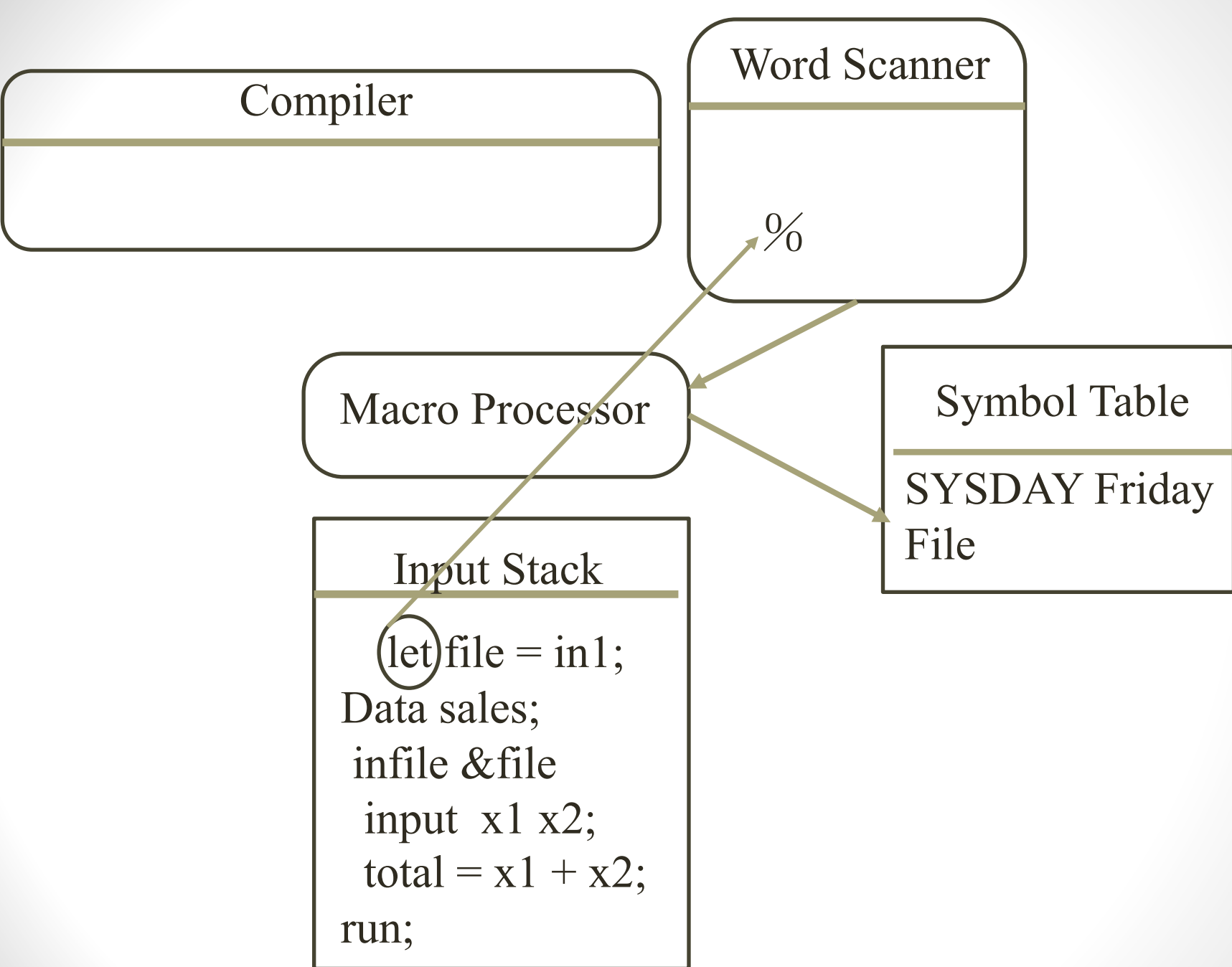
Macro Processor

Input Stack

```
%let file = in1;  
Data sales;  
infile &file  
input x1 x2;  
total = x1 + x2;  
run;
```

Symbol Table

SYSDAY Friday



Compiler

Word Scanner

Data sales

Macro Processor

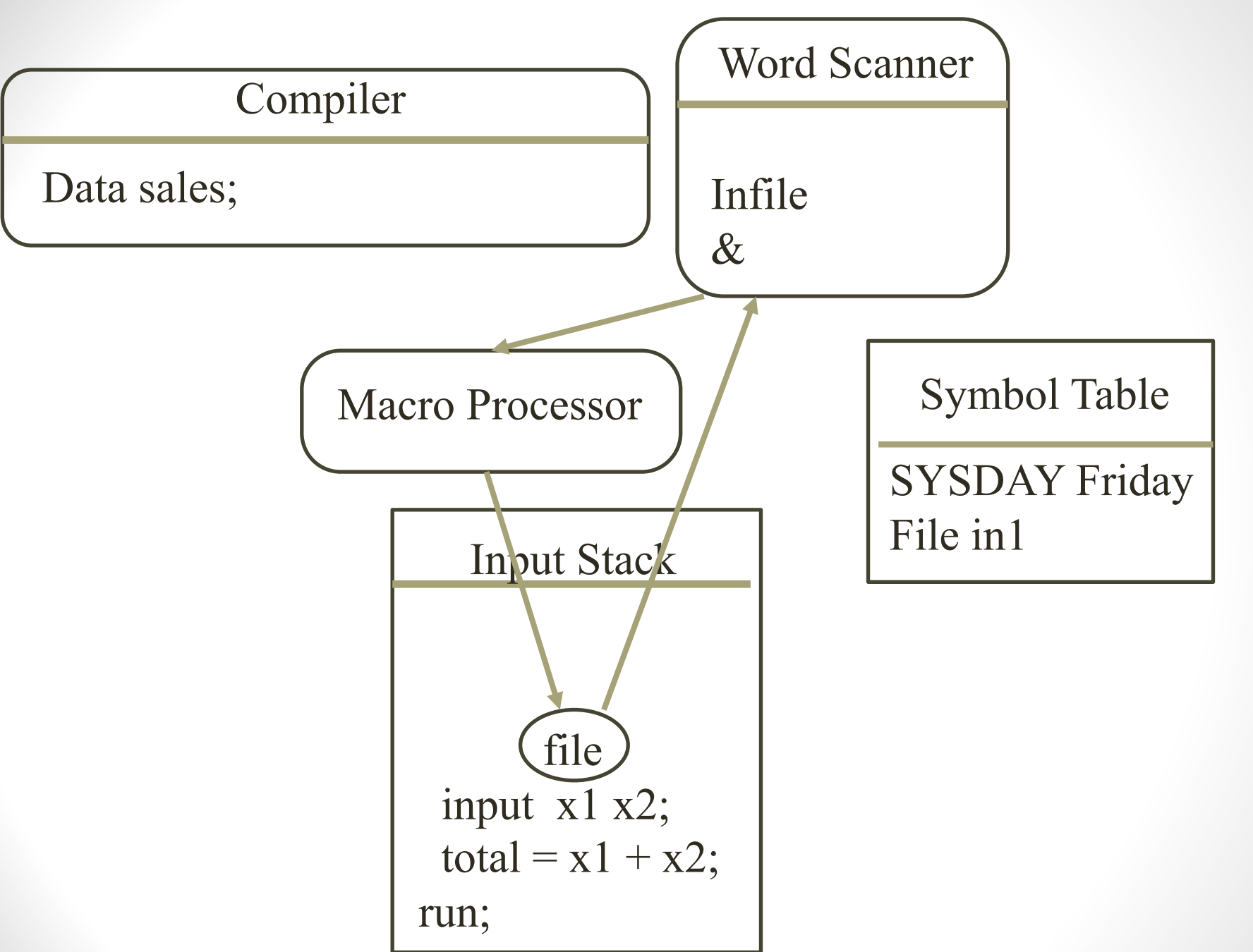
Symbol Table

SYSDAY Friday  
File in1

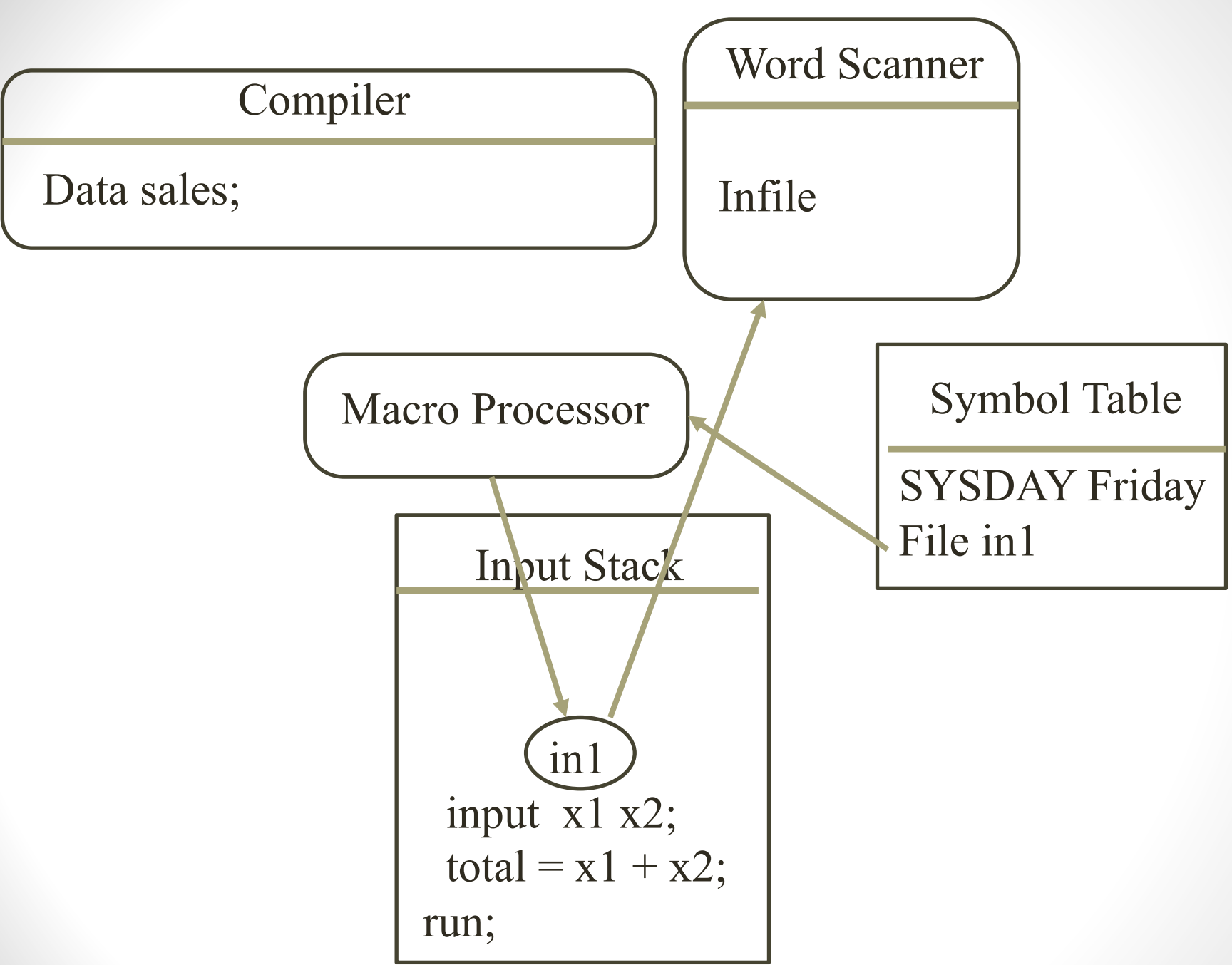
Input Stack

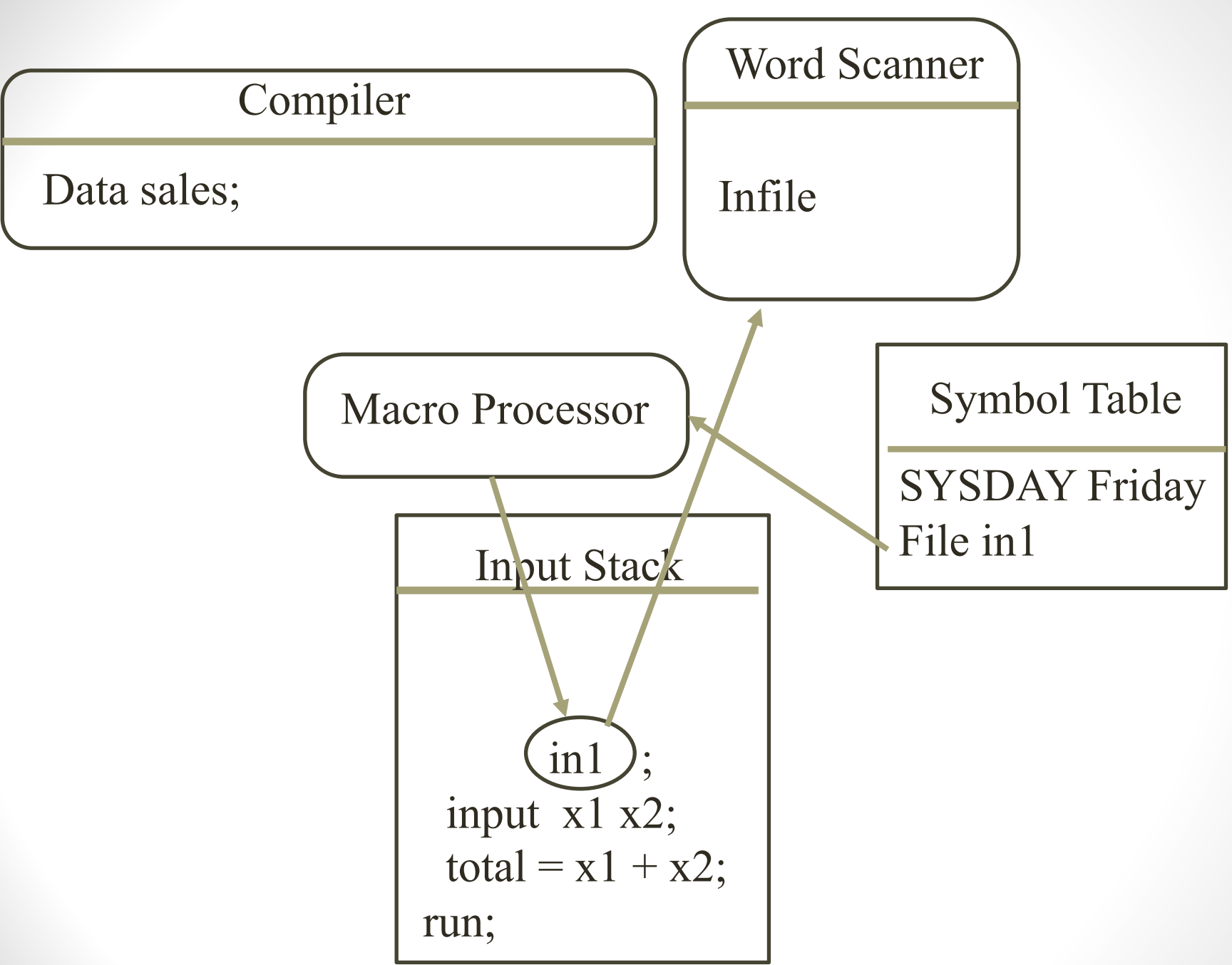
;

infile &file  
input x1 x2;  
total = x1 + x2;  
run;









## Compiler

Data sales;  
Infile in1;  
Input x1 x2;  
Total = x1 + x2

## Word Scanner

run  
;

## Macro Processor

## Symbol Table

SYSDAY Friday  
File in1

## Input Stack

# 巨集變數

- 巨集變數為動態 SAS 變數
- 內容可以是數字，變數，字串等
- 變數的數值最多可以有 65534 字元
- 巨集變數的長度等於所設定文字的長度
- 設定方式
  - 使用者設定
  - SAS 自動產生
- 屬性
  - Local – 僅定義在某巨集程式
  - Global – 目前 SAS 程式

# 巨集變數名稱

- 巨集變數開始需為字母 a-z 或底線，接續可為 a-z、底線或是 0-9
- 字頭 AF、DMS、SQL 與 SYS 不建議使用
- 最多可以 32 字元
- 不可以有空白
- 不可以有底線以外的特殊符號
- 不區分大，小寫

# 巨集變數內容設定方式

| 設定                 | 範例                                                                                                                                                                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 文字 (constant text) | <code>%let street = maple;</code><br>不需要加引號，若有引號會變成巨集變數數值的一部分                                                                                                       |
| 數字                 | <code>%let num = 123;</code>                                                                                                                                        |
| 數學表示式              | 利用 <code>%EVAL</code> 函數<br><code>%let num=%eval(100+200);</code>                                                                                                   |
| 空值 (Null)          | <code>%let country=;</code>                                                                                                                                         |
| 包含巨集變數             | <code>%let street = Maple;</code><br><code>%let num = 123;</code><br><code>%let address=&amp;num &amp;street Avenue;</code><br><code>/* 123 Maple Anvenue */</code> |
| 空白或是特殊字元           | 利用 <code>%STR</code> 或 <code>%NRSTR</code> 設定                                                                                                                       |
| 從已存在資料集取得數據        | 利用 <code>SYMPUT</code> 取得資料數值                                                                                                                                       |

# 引用巨集變數

- 引用巨集變數方式，會在變數前加 &
- 引用時須使用雙引號
- 範例

`%let dsn = Newdata;`

`Title1 “Contents of Data Set &dsn”;` (正確)

`Title2 ‘Contents of Data Set &dsn’;` (錯誤)

# 合併巨集變數與文字

- 文字可以與巨集變數整合

```
%let name=sales;  
data new&name;  
set save.&name;  
more SAS statements  
if units>100;  
run;
```

```
DATA NEWSALES;  
SET SAVE.SALES;  
more SAS statements  
IF UNITS>100;  
RUN;
```



# 必要語法

| 語法                                                                                            | 説明   |
|-----------------------------------------------------------------------------------------------|------|
| <code>%MACRO macro-name &lt;(parameter-list)&gt;&lt;/option-1 &lt;...option-n&gt;&gt;;</code> | 巨集開始 |
| <code>%MEND macro-name;</code>                                                                | 巨集結束 |

範例

```
%macro one;
```

巨集内容

```
%mend one;
```

# 範例 – 設定輸入參數

```
%macro prnt(var,sum);  
  proc print data=ch9d1;  
    var &var;  
    sum &sum;  
  run;  
%mend prnt;
```

# 範例 – 輸入參數使用關鍵參數

```
%macro prnt(xvar = var, yvar = sum);  
  proc freq data=ch9d1;  
    tables &xvar*&yvar;  
  run;  
%mend prnt;
```

# MACRO 選項

| 選項                                | 說明                                 |
|-----------------------------------|------------------------------------|
| MINDELIMITER = 'single character' | 修改分隔方式<br>但 %、&、'、”、()、; 不可設定為分隔符號 |
| MINOPERATOR   NOMINOPERATOR       | 設定巨集可以使用助記符號<br>(mnemonic) IN      |

# 巨集語法

| 語法      | 說明                              |
|---------|---------------------------------|
| %GLOBAL | 產生整合性巨集變數                       |
| %LET    | 新建巨集變數 -- 設定數值或字串<br>改變原本巨集變數內容 |
| %LOCAL  | 產生區域性巨集變數                       |
| %PUT    | 將巨集變數數值列印到 SAS 日誌               |

# 呈現巨集變數數據的方式

- `%let a=first;`
- `%let b=macro variable;`
- `%put &a ***&b***; /* 輸出在 log */`

# %LET

## Syntax

**%LET** *macro-variable* =<value>;

For example

**%macro** title(text,number);

title&number "&text";

**%mend**;

**%let** topic= DUSTY; /\* Leading and trailing \*/ /\*  
blanks are removed \*/

**%title**(&topic,1)

# %LOCAL

## Syntax

**%LOCAL** *macro-variable(s)*;

- ***macro-variable(s)***
  - 一個或多個巨集變數
  - 不可以使用一序列的 SAS 變數 (variable list)



# 範例

```
%let variable=1;  
%macro routine;  
    %put ***** Beginning ROUTINE *****;  
    %local variable;  
    %let variable=2;  
    %put The value of variable inside ROUTINE is  
        &variable;  
    %put ***** Ending ROUTINE *****;  
%mend routine;  
%routine;
```

# %GLOBAL

## Syntax

**%GLOBAL** *macro-variable(s)*;

## Example

```
%macro vars(first=1,last=);
```

```
    %global gfirst glast;
```

```
    %let gfirst=&first;
```

```
    %let glast=&last;
```

```
    var q&first-q&last;
```

```
%mend vars;
```

# 範例

```
%let county=Clark;  
%macro concat;  
data _null_;  
    length longname $20;  
    longname("&county"||" County");  
    put longname;  
run;  
%mend concat;
```

# %PUT

## Syntax

**%PUT** <*text* | `_ALL_` | `_AUTOMATIC_` |  
`_GLOBAL_` | `_LOCAL_` | `_USER_`>;

## Example

- `%put One line of text.;`
- `%put _automatic_;`

# 範例

```
%macro holinfo(day,date);  
  %let holiday=Christmas;  
  %put *** Inside macro: ***;  
  %put *** &holiday occurs on &day, &date, 2002.  
  ***;  
%mend holinfo;  
%holinfo(Wednesday,12/25);
```

# 範例

```
%macro compare(first,second);  
  %if &first>&second %then  
    %put &first > &second;  
  %else %if &first=&second %then  
    %put &first = &second;  
  %else %put &first<&second;  
%mend compare;
```

# 指令 -- 判定語法

| 語法              | 説明                                                                                    |
|-----------------|---------------------------------------------------------------------------------------|
| %DO             | Treats text and program statements as a unit until a matching %END statement appears. |
| %IF %THEN %ELSE | Executes a portion of a macro conditionally.                                          |
| %END            | Ends a %DO group.                                                                     |

# 範例

```
%macro settax(taxrate);  
  %if &taxrate = CHANGE %then  
    %do;  
      data thisyear;  
        set lastyear;  
        if sale > 100 then tax = .05;  
        else tax = .08;  
    run;  
  %end;  
%else %if &taxrate = SAME %then  
  %do;  
    data thisyear;  
      set lastyear;  
      tax = .03;  
    run;  
  %end;  
%mend settax;
```



# 巨集函數

%EVAL ( *expression* );

%SYSEVALF( *expression,conversion-type* )

# 算式

- 巨集主要處理字串
- 巨集可以進行簡單運算，使用 `%EVAL`，但以整數為主
- 若需要浮點，則需利用 `%SYSEVALF` 函數

# 算式 %EVAL

- `%let a=%eval(1+2);`
  - `%let b=%eval(10*3);`
  - `%let c=%eval(4/2);`
  - `%let i=%eval(5/3);`
  - `%put The value of a is &a;`
  - `%put The value of b is &b;`
  - `%put The value of c is &c;`
  - `%put The value of I is &i;`
- The value of a is 3.
  - The value of b is 30.
  - The value of c is 2.
  - The value of I is 1.

# 算式 %SYSEVALF

- `%let a=%sysevalf(10.0*3.0);`      •  $10.0 * 3.0 = 30$
- `%let b=%sysevalf(10.5+20.8);`      •  $10.5 + 20.8 = 31.3$
- `%let c=%sysevalf(5/3);`              •  $5/3 = 1.66666666667$
- `%put 10.0*3.0 = &a;`
- `%put 10.5+20.8 = &b;`
- `%put 5/3 = &c;`

# 邏輯表示式

- 整數

```
%macro compnum(first, second);
```

```
    %if &first > &second %then
```

```
        %put &first is greater than &second;
```

```
    %else %if &first = &second %then
```

```
        %put &first equals &second;
```

```
    %else %put &first is less than &second;
```

```
%mend compnum;
```

# 邏輯表示式

- 浮點

```
%macro compflt(first,second);  
    %if %sysevalf(&first > &second) %then  
        %put &first is greater than &second;  
    %else %if %sysevalf(&first = &second) %then  
        %put &first equals &second;  
    %else %put &first is less than &second;  
%mend compflt;
```

# 遞迴語法

| 語法            | 說明                |
|---------------|-------------------|
| iterative %Do | 利用指標變數讓巨集程式重複執行   |
| %DO %UNTIL    | 重複執行巨集程式直到設定條件符合  |
| %DO %WHILE    | 重複執行巨集程式直到不符合設定條件 |
| %END          | 結束                |

# ITERATIVE %DO

## 語法

```
%DO macro-variable=start %TO stop <%BY increment>;  
    text and macro language statements  
%END;
```

## 説明

```
%macro create(howmany);  
    %do i=1 %to &howmany;  
        data month&i;  
        infile in&i;  
        input product cost date; run;  
    %end;  
%mend create;  
%create(3)
```



# %DO %WHILE (%UNTIL)

## 語法

%DO %WHILE (*expression*);  
    *text and macro program statements*

%END;

%DO %UNTIL (*expression*);  
    *text and macro language statements*

%END;

# 範例

```
%macro test(finish);  
    %let i=1;  
    %do %while (&i<&finish);  
        %put the value of i is &i;  
        %let i=%eval(&i+1);  
    %end;  
%mend test;
```

# 巨集引用 (Quoting)

- 巨集是以字元為基礎的語言
- 巨集程序可以處理任何特殊字元
- 但因巨集語法會需要用到部分特殊字元
- 為避免造成語法與特殊字元混淆
- 需要使用 **macro quoting** 函數
- 需要特別處理的特殊字元

|           |   |     |    |
|-----------|---|-----|----|
| blank     | ) | =   | LT |
| ;         | ( |     | GE |
| ¬         | + | AND | GT |
| ^         | — | OR  | IN |
| ~         | * | NOT | %  |
| , (comma) | / | EQ  | &  |
| '         | < | NE  | #  |
| "         | > | LE  |    |

# Macro Quoting functions

- 兩種層級執行方式
- **Compilation (編輯)**
  - 編輯過程遮蔽特殊符號
  - `%let printit = %str(proc print; run;);`
  - `%str` 避免 `%let` 提早終止
  - 保留 ; 在 `PROC PRINT`
- **Execution (執行)**
  - 執行過程解析特殊符號
  - `%if %bquote(&state) = nc %then %put North Carolina Dept. of Revenue;`
  - `%bquote` 避免 `&state` 解析 OR (Oregon 的縮寫) 為邏輯判定式 OR

# 巨集引用特殊符號

| Group | Items                                                         | Macro Quoting Functions                                                  |
|-------|---------------------------------------------------------------|--------------------------------------------------------------------------|
| A     | + — */<>=¬^ ~;; #<br>blank AND OR NOT EQ<br>NE LE LT GE GT IN | all                                                                      |
| B     | &%                                                            | %NRSTR, %NRBQUOTE,<br>%SUPERQ, %NRQUOTE                                  |
| C     | unmatched' “()                                                | %BQUOTE, %NRBQUOTE,<br>%SUPERQ, %STR*,<br>%NRSTR*, %QUOTE*,<br>%NRQUOTE* |

# MACRO FUNCTIONS -- Execution

| Function  | Groups | 說明                    |
|-----------|--------|-----------------------|
| %BQUOTE   | A、C    | 執行階段避免解析特殊符號          |
| %NRBQUOTE | A、B、C  |                       |
| %QUOTE    | A、C*   | 執行階段避免解析特殊符號<br>不對稱引號 |
| %NRQUOTE  | A、B、C* |                       |
| %SUPERQ   | A、B、C  | 執行階段避免解析特殊符號，但不解析數值   |

NR: Note resolved

# MACRO FUNCTIONS --

## Compilation

| Function | Group  | 說明         |
|----------|--------|------------|
| %NRSTR   | A、B、C* | 編輯過程遮蔽特殊符號 |
| %STR     | A、C*   |            |

# %QUOTE

## 語法

**%QUOTE** (*character string* | *text expression*)

## 範例

```
%macro dept2(state);  
    /* with %quote function--problems are prevented */  
    %if %quote(&state)=nc %then  
        %put North Carolina Department of Revenue;  
    %else %put Department of Revenue;  
%mend dept2;  
  
%dept2(or)
```



# %STR

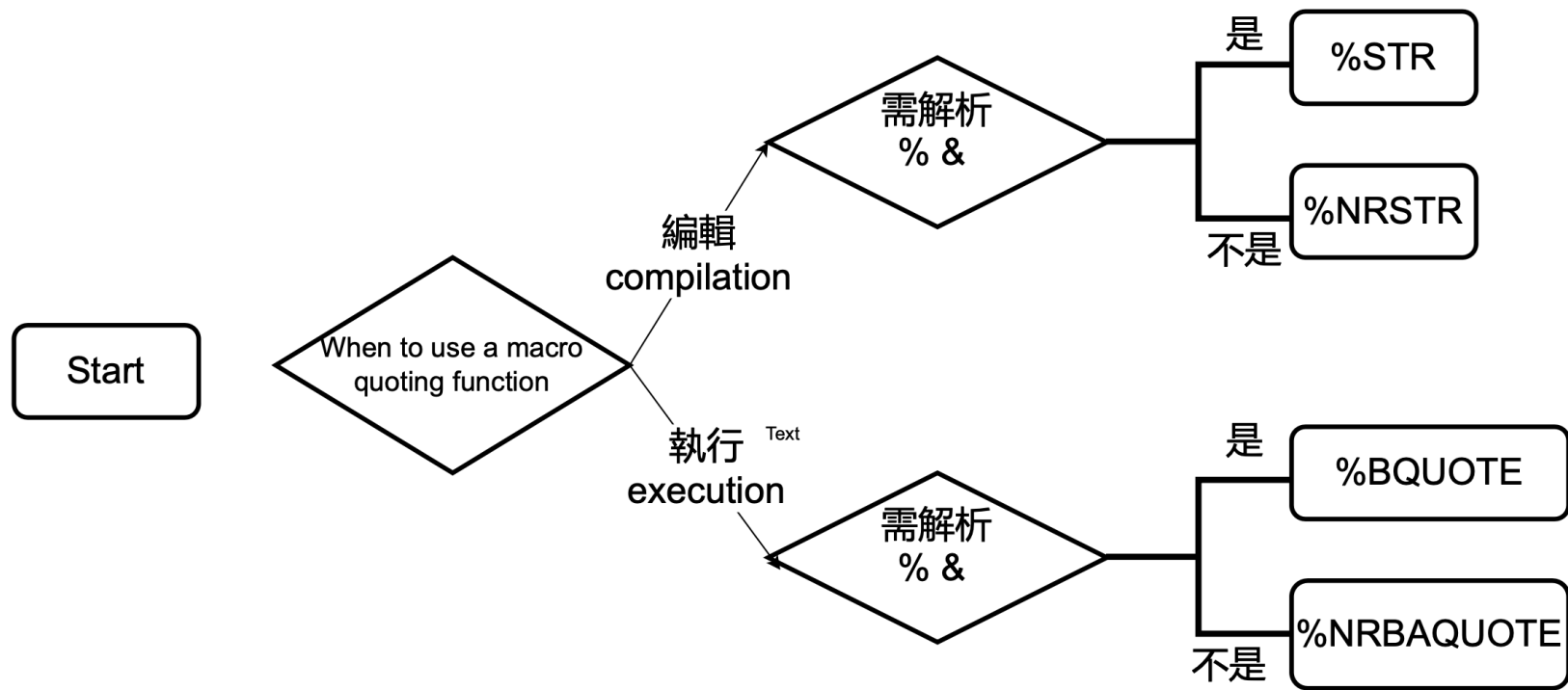
## Syntax

**%STR** (*character-string*)

例如

```
%let time=%str(  now); /*contain leading blanks. */  
%put Text followed by the value of time:&time;
```

# 引用



```
%let name = %nrquote(Tom&Jerry);  
%let footer1 = %nrquote(%percent = n/N);
```

Log will put warning as blew.

```
1      %let name = %nrquote(Tom&Jerry);  
WARNING: Apparent symbolic reference JERRY not resolved.  
2      %let footer1 = %nrquote(%percent = n/N);  
WARNING: Apparent invocation of macro PERCENT not resolved.
```

# 範例

- `%let name =  
%NRSTR(Tom&Jerry);`
- \*解析 Ampersand  
(%)
- `%let cartoon1 =  
%STR(&name.);`
- `%put %cartoon1;  
Tom&Jerry`

- \*不解析 Ampersand
- `%let cartoon2 =  
%NRSTR(&name.);`
- `%put &cartoon2 .;  
&name .`

# MACRO FUNCTIONS

| Function | Description |
|----------|-------------|
| %INDEX   | 找出某段字串出現的位置 |
| %LENGTH  | 提供字串長度      |
| %UPCASE  | 將字串轉成大寫     |

# %UPCASE

## 語法

`%UPCASE` (*character string* | *text expression*)

## 例如

```
%macro runrept(month);  
  %if %upcase(&month)=DEC  
    %then %str(proc fsview  
      data=reports.endyear; run;);  
  %else %str(proc fsview  
    data=reports.&month; run;);  
%mend runrept;
```

# %INDEX

## Syntax

**%INDEX** (*source*, *string*)

## Note

- ***source***
  - 原始字串
- ***string***
  - 尋找的字串

# 範例

```
%let a=a very long value;  
%let b=%index(&a,v);  
%put V appears at position &b .;
```



# %LENGTH

## Syntax

**%LENGTH** (*character string* | *text expression*)

## 範例

```
%let a=Happy;
```

```
%let b=Birthday;
```

```
%put The length of &a is %length(&a).;
```

```
%put The length of &b is %length(&b).;
```

```
%put The length of &a &b To You is %length(&a &b to  
you).;
```

# MACRO FUNCTIONS

| Function | Description |
|----------|-------------|
| %SUBSTR  | 分割字串        |
| %SCAN    | 擷取有規則分隔的字串  |

# %SUBSTR

語法

%SUBSTR (*argument*, *position*<, *length*>)

- *argument*
  - 原始字串
- *position*
  - 分割字串的起始位置
- *length*
  - 分割字串的長度

# 範例

```
%macro makefref(fileref,file);  
  %if %length(&fileref) gt 8 %then  
    %let fileref = %substr(&fileref,1,8);    filename  
    &fileref "&file";  
%mend makefref;  
%makefref(humanresource,/dept/humanresource/report96)
```

# %SCAN

## 語法

`%SCAN(argument,n<,delimiters>)`

- Argument

- 字串或文字表示式
- 包含特殊字元會與 %QUOTE 並用

- N -- 整數表示欲擷取字串的位置

- Delimiters

- 預設分隔符號
- ASCII systems -- blank . < ( + & ! \$ \* ) ; ^ - / , % |
- EBCDIC systems -- blank . < ( + | & ! \$ \* ) ; ~ - / , % | ¢
- 通常會與 %STR 並用

# 範例

```
%let listname=ars qm vmct;  
%let var=%scan(%quote(&listname),1);
```

# DATA STEP INTERFACES

| 語法     | 說明                              |
|--------|---------------------------------|
| SYMGET | 回傳巨集變數的數值到 DATA                 |
| SYMPUT | 由 DATA 所提供的數據來產生新巨集變數或取代巨集變數的數值 |

# 範例

```
data dusty;  
  input dept $ name $ salary @@;  
  cards;  
bedding Watlee 18000   bedding Ives 16000  
bedding Parker 9000   bedding George 8000  
bedding Joiner 8000   carpet Keller 20000  
carpet Ray 12000      carpet Jones 9000  
gifts Johnston 8000   gifts Matthew 19000  
kitchen White 8000    kitchen Banks 14000  
kitchen Marks 9000    kitchen Cannon 15000  
tv Jones 9000         tv Smith 8000  
tv Rogers 15000       tv Morse 16000  
run;
```



# 範例

```
proc means noprint;  
    class dept;  
    var salary;  
    output out=stats sum=s_sal;  
Run;  
data _null_;  
    set stats;  
    if _n_=1 then call symput('s_tot',s_sal);  
    else call symput('s'||dept,s_sal);  
run;
```

# 範例

```
data new;  
set dusty;  
pctdept=(salary/symget('s' || dept))*100;  
pcttot=(salary/&s_tot)*100;  
pcttot1=(salary/symget('s_tot'))*100;  
run;
```

# 範例 – 儀器使用情況

- statpackch8\_1.xls – statpackch8\_5.xls 提供某醫院 5 個月某項監測儀器使用的情況

| 欄位 | 變數          |      |
|----|-------------|------|
| A  | DEPT_NO     | 科別   |
| B  | CREATE_USER | 使用者  |
| C  | CREATE_DATE | 使用時間 |
| D  | VS_ITEM     | 使用項目 |
| E  | VS_TYPE     | 使用類型 |
| F  | VIP_FLAG    | VIP  |

- 每個檔案有三個表單 Sheet1 – Sheet3